

MARS: A Safety-Filtered Multi-Robot Area-Coverage Testbed with Classical, Random-Walk and MAPPO Controllers

Harsh Pandhe
Independent Research
harshpandhehome@gmail.com

Abstract—We describe MARS (Multi-Agent Robot Swarm), an open ROS 2 Jazzy / Gazebo Harmonic testbed for cooperative area coverage by a small team of differential-drive robots. It puts three controllers behind one three-stage safety stack (two fixed override rules, a soft control-barrier-function filter, and a front safeguard): a classical decentralized planner (frontier selection, A* routing, Voronoi partitioning with consensus-based auction claims within a 3 m communication range, and a behavior tree), a random walk, and a MAPPO policy with a permutation-invariant neighbor encoder and a centralized critic, across five Gazebo worlds. In long single reference runs the classical planner reached high discovered-map ACR (85.7–100% in four worlds, 25.0% in the maze), and in an exploratory single-episode matrix a trained MAPPO checkpoint had lower observed discovered-map ACR than the classical planner in all five worlds; the comparison is between an engineered system and a minimally trained reactive policy, not a matched-information benchmark. We also document three limits of the testbed: evaluation of an identical checkpoint varies between runs (a documented property that users must account for), the safety margin is soft with no formal guarantee, and the architecture’s transfer to other team sizes is untested. Code and per-run results for the reported experiments are archived with a DOI.

Index Terms—multi-robot systems, area coverage, control barrier functions, MAPPO, reproducibility

I. INTRODUCTION

Cooperative area coverage, in which a team of robots jointly sweeps an unknown environment, underlies search, inspection and monitoring. Evaluating coordination methods for it needs a simulator that models range sensing and contact, and results that others can reproduce. MARS is a testbed built for that purpose. It wraps Gazebo in a multi-agent environment, provides a classical decentralized controller and a learned MAPPO baseline behind the same safety stack, and archives per-run results.

Contributions.

- A documented testbed with five worlds, three controllers and a common safety stack, described here as implemented (Sec. III).
- Long-run reference coverage of the classical controller in the five worlds, a robot-count sweep, and a comparison with trained MAPPO baselines, all from archived runs (Sec. IV).

- An account of the testbed’s limits: non-deterministic evaluation, a soft safety margin, and claims that remain untested (Sec. V).

We present the baselines as a starting point, not as state-of-the-art ceilings, and leave open for users the questions of competitive learned policies, a safety filter with a formal guarantee, and transfer across team sizes.

II. RELATED WORK

Frontier-based exploration [1] and its multi-robot extensions [2], [3] are the classical baseline for this task; the consensus-based bundle algorithm [4] supplies the auction mechanism we use to resolve duplicate target claims. MAPPO [5] is a strong baseline for cooperative multi-agent learning; we train it with RLlib [6]. The actor’s neighbor encoder is a mean-pooled embedding in the style of Deep Sets [7]; it contains no message passing, so we do not call it a graph neural network. Control barrier functions [8] give a quadratic-program safety filter, and barrier-function filters have been placed around learned policies [9]. We solve ours with OSQP [10] and add a slack variable, which makes it soft (Sec. III-C). A companion study [11] examines the MAPPO baseline in depth.

III. THE TESTBED

A. Platform and Metrics

TurtleBot3 Waffle robots (differential drive, 24-sector LiDAR, odometry), instantiated in teams of $N \in \{2, 3, 5, 8\}$ for the classical-controller sweep and $N = 3$ for the learned experiments, run in Gazebo Harmonic under ROS 2 Jazzy [12], [13]. The simulator is wrapped as a PettingZoo [14] parallel environment, so any code that speaks the standard multi-agent parallel API can drive the simulator without touching ROS 2 internals; our own training uses it through RLlib. The workspace is a grid of 0.4 m cells. A cell is marked visited when a robot’s pose falls in it or a LiDAR ray (up to 2.0 m) passes through it as free space. The *area-coverage rate* is

$$\text{ACR} = \frac{|\text{visited} \setminus \text{obstacle}|}{N_{\text{cells}} - |\text{obstacle}|} \times 100, \quad (1)$$

where “obstacle” cells are those the robots have registered as obstacles, so the denominator depends on what has been discovered. ACR here is therefore a *discovered-map* metric: controllers that discover obstacles at different rates have different denominators, so ACR is not a fixed-denominator measure of physical coverage, and a controller that discovers few obstacles keeps a larger denominator (lowering its ACR), while one that finds obstacles quickly, for example by hugging walls, shrinks it early. A ground-truth-denominator metric is not reported (Sec. V). A *wall-proximity event* is counted when the minimum LiDAR range is below 0.14 m (the approximate sensor-to-bumper distance); an *inter-robot proximity event* when two robots are closer than 0.20 m after the first 15 steps. Events are counted per robot per control step. These are threshold-based sensor proxies, not verified physical contacts. The inter-robot threshold is smaller than the filter’s separation buffer (Sec. III-C), so a zero count does not show that the margin held.

B. Observation and Action

Each robot observes a 46-vector: 24 LiDAR sector ranges, goal distance and bearing, linear and angular velocity, and nine neighbor slots of (distance, bearing), padded with (10.0, 0.0). Actions are (v, ω) with $|v| \leq 0.22$ m/s and $|\omega| \leq 1.0$ rad/s. Because the vector has a fixed length, the network accepts up to nine neighbors.

C. Safety Stack

Every controller’s command passes through three stages. (1) *Override rules*. If the minimum LiDAR range over all beams is below 0.22 m and the commanded forward speed is positive, forward speed becomes -0.08 m/s; if another robot is within 0.45 m and within 72° of the heading, forward speed becomes -0.05 m/s. (2) *Soft CBF*. A quadratic program (OSQP) solves

$$\begin{aligned} \min_{v, \omega, \delta} & (v - v_{\text{nom}})^2 + 0.05(\omega - \omega_{\text{nom}})^2 + P\delta^2 \\ \text{s.t.} & A_k v + B_k \omega + \delta \geq -\gamma h_k, \end{aligned} \quad (2)$$

with $-0.22 \leq v \leq 0.22$, $-1.0 \leq \omega \leq 1.0$ (the action bounds, enforced as variable bounds), $0 \leq \delta \leq 10$, $P = 1000$ and $\gamma = 2.0$. Barrier terms come from LiDAR returns within 0.6 m, evaluated at two lookahead points (± 0.12 m on the heading axis) with obstacle radius 0.20 m, and from neighbors within 1.2 m with a separation buffer of 0.45 m (not a physical radius). The neighbor term treats the neighbor’s velocity as bounded and unmodeled. If the QP is infeasible the filter returns $(\min(0, v_{\text{nom}}), \omega_{\text{nom}})$. (3) *Front safeguard*. Forward speed is set to $\min(0, v)$ if any of the five forward-cone beams reads below 0.24 m. The bounded slack can relax conflicting constraints when the required relaxation is within its bound (otherwise the QP is infeasible and the fallback above is used), at the price of the guarantee: because of δ , the radii are soft constraints: we claim no formal guarantee and report behaviour only.

D. Classical Controller

Each robot keeps its own visited and obstacle grids, built from its LiDAR. The stack has these components:

- **Map sharing:** robots within 3.0 m merge visited grids by bitwise OR, a range-limited approximation of gossip consensus; a global grid, accumulated from all robots’ LiDAR returns and not exposed to the controllers, is kept only for monitoring and for computing ACR; its obstacle set is the ACR denominator’s obstacle set.
- **Target selection:** frontier cells are the nearest unvisited candidates inside the robot’s own Voronoi cell relative to sensed neighbors; duplicate claims are resolved by a consensus-based auction among peers in range, and a stale claim from a peer that has left range expires after 30 steps. Unreachable targets are blacklisted.
- **Routing:** 8-connected A* on an obstacle-inflated grid.
- **Behavior tree:** in priority order, mission-complete standby or perimeter patrol, an escape maneuver ($v = -0.12$ m/s, $\omega = \pm 0.6$ rad/s) when a robot has not moved for 30 steps, frontier exploration, and a boundary-aware wander.
- **Localization:** a correlative scan matcher corrects odometry drift.

E. Random-Walk Controller

At each step every robot’s command is drawn uniformly from the action box $[-0.22, 0.22] \times [-1.0, 1.0]$ (the environment’s `action_space.sample()`), independently per robot and step, with no observation or goal information, and then passes through the same three-stage safety stack. The Gazebo seed is fixed (42); no separate seed for the action sampler is recorded.

F. MAPPO Baseline

A single policy is shared by all robots. Each neighbor’s (distance, bearing) pair is embedded by a two-layer ReLU network ($2 \rightarrow 16 \rightarrow 16$), masked and mean-pooled, concatenated with the 28 ego features and passed through a two-layer Tanh actor ($44 \rightarrow 64 \rightarrow 64$). A centralized critic ($76 \rightarrow 64 \rightarrow 64 \rightarrow 1$, Tanh) takes the same 44 features plus a mean-pooled embedding ($48 \rightarrow 32 \rightarrow 32$) of the other agents’ observation-action pairs. The mean-pooled embedding is invariant to neighbor order and padding position, which `tests/test_permutation_invariance.py` checks directly. Training uses RLlib PPO [15] with a single in-process rollout worker, train batch 700, 90-step episodes, minibatch 64, 5 epochs, learning rate 10^{-4} , clip 0.2, $\gamma = 0.99$, GAE $\lambda = 0.95$ and entropy coefficient 0.01. The per-step reward is

$$\begin{aligned} r_i = & -0.01 + 5.0 \Delta d_i + 4.0 n_{\text{new}} + 20 \mathbf{1}[\text{goal}] \\ & - 8.0 \mathbf{1}[\text{wall}] - \rho_i, \end{aligned} \quad (3)$$

where Δd_i is the reduction in goal distance, n_{new} the number of cells newly visited by any robot this step, and $\rho_i = 0.5 \mathbf{1}[d_{\min, i} < 0.5 \text{ m}] + 10 \mathbf{1}[d_{\min, i} < 0.20 \text{ m}]$ with $d_{\min, i}$ the distance to the nearest other robot. The discovery term

TABLE I

CLASSICAL CONTROLLER, THREE ROBOTS, ONE LONG RUN PER WORLD (SINGLE-RUN REFERENCE VALUES, NOT AN ESTABLISHED BENCHMARK). PROXIMITY EVENTS ARE WALL / INTER-ROBOT COUNTS (THRESHOLD PROXIES, NOT VERIFIED CONTACTS) OVER THE WHOLE RUN; CLEARANCE IS THE MINIMUM INTER-ROBOT DISTANCE.

World	Cells	Steps	ACR	Plateau	Dist. (m)	Wall	Agent	Clr. (m)
warehouse	9,375	9,050	100.0	7,660	1263	17,018	0	0.81
office	3,500	3,500	98.2	3,466	377	0	0	0.38
depot	3,000	3,500	85.7	2,906	180	477	0	0.55
cafe	800	1,200	100.0	397	82	738	0	0.46
maze	4,800	4,000	25.0	3,961	316	1,108	0	0.26

n_{new} is a team-level quantity used only in the training reward; it is not part of the decentralized observation. The goal is a random point from a fixed per-world list of safe positions, reassigned on arrival; it is less informed than the classical controller’s map-aware frontier targets. The policy is purely reactive: it has no recurrent state or local map, and no spatial-memory variant was trained. Evaluation uses greedy actions. All learned results use a single archived checkpoint (checkpoints/mappo_baseline, trained on the cafe); the training seed and iteration count of that checkpoint are not recorded in its archived metadata and no checkpoint-selection procedure was applied, so no across-seed estimate is claimed for it. The checkpoint is identified by the SHA-256 of its policy state, f07518b9...ab6edb.

IV. RESULTS

Protocol and caveats. Unless stated, each cell is one episode. We found that repeating an evaluation of an identical checkpoint and seed gives different outcomes (Sec. V), so single-episode numbers carry unquantified run-to-run variance; we treat them as descriptive. Raw per-run records are archived (Data availability).

A. Long-Run Reference Runs of the Classical Controller

Table I reports one long run per world with three robots. The step budget differs by world and was chosen to see where coverage levels off, so “steps to plateau” is the first sampled step at which ACR came within 0.5 points of its final value; it is not a controller-versus-controller comparison.

In 3 of 5 worlds (warehouse, depot, cafe) ACR reached its plateau before 90% of the run, so those budgets were long enough to see it level off; for the others the final value may still have been rising. The maze’s figure should not be read as a success or a failure by itself: we did not measure how much of its bounding box is traversable, and the ACR denominator only excludes cells where a LiDAR return has registered an obstacle. The warehouse run recorded a very large wall-proximity count over its long perimeter patrol after coverage saturated, which the proximity metric (sensor range below 0.14m) makes easy to accumulate.

B. Team Size

Table II varies the number of robots from 2 to 8 in each world for 200 steps (one run per cell). Coverage generally

TABLE II

EXPLORATORY SINGLE-RUN BASELINE: CLASSICAL CONTROLLER, 200 STEPS, ONE RUN PER CELL, DISCOVERED-MAP ACR % (INTER-ROBOT PROXIMITY EVENTS).

World	$N=2$	$N=3$	$N=5$	$N=8$
warehouse	3.4 (0)	4.8 (0)	5.3 (0)	8.7 (0)
office	5.9 (0)	6.1 (0)	12.7 (0)	46.8 (2)
depot	16.9 (0)	16.6 (0)	31.9 (0)	54.4 (0)
cafe	64.3 (0)	69.3 (0)	66.9 (2)	98.6 (8)
maze	1.1 (0)	3.2 (0)	6.0 (0)	10.0 (0)

TABLE III

EXPLORATORY SINGLE-EPISEODE BASELINE: DISCOVERED-MAP ACR % AFTER 150 STEPS, THREE ROBOTS, ONE EPISODE PER CELL. MAPPO RESULTS ARE EVALUATIONS OF A SINGLE TRAINED CHECKPOINT AND SERVE AS AN EXPLORATORY BASELINE (SEE SEC. III).

World	Classical	Random	MAPPO
cafe	57.0	22.6	28.2
warehouse	4.3	2.6	3.3
depot	13.3	6.8	6.7
office	3.8	1.7	3.0
maze	2.7	1.5	2.2

rose with team size, though not monotonically (cafe, $N=3 \rightarrow 5$), and the 200-step horizon is far from the plateau in the larger worlds. Inter-robot proximity events occurred in 3 of 20 configurations, all at $N \geq 5$ in the two smallest arenas (cafe and office). No other explanation for that pattern was tested.

C. Learned versus Classical

Table III compares the trained MAPPO policy (trained on the cafe, zero-shot elsewhere) with the classical controller and a random walk, one 150-step episode per cell with three robots. In this single-episode matrix the classical controller had higher observed discovered-map ACR than MAPPO in all five worlds. This single-episode matrix is exploratory; the companion study [11] reports a 10-episode cafe comparison in which MAPPO’s median coverage (14.5%) was below both the classical controller’s (38.6%) and a random walk’s (29.6%); the per-episode data for that comparison are not archived. Because the controllers differ in information (the classical one builds a map, MAPPO sees only local ranges and a random goal), this compares an engineered system with a minimally trained learned one, not two learned policies.

The reward coefficients are exposed as environment variables, which allows reward-shaping ablations without editing the environment; an exploratory ablation of three variants (three training seeds each, no variant significantly different at $p < 0.05$) is analyzed in the companion study [11].

D. Safety Observations

Across these heterogeneous exploratory runs (20 sweep runs, 25 matrix runs and a 27-episode ablation, 72 in total), inter-robot proximity events were absent in 69 of 72, while coverage ranged from 1.1% to 98.6% (across worlds and horizons), so the event count did not distinguish weak

from strong controllers. In the office run the minimum inter-robot distance was 0.38 m, below the 0.45 m CBF separation buffer, with no proximity event, consistent with the intended behaviour of a soft safety stack. Two one-off qualitative stress tests with moving hazards (head-on at 0.38 m/s and crossing at 0.22 m/s) were run once each without a proximity event; their raw traces are not archived.

V. LIMITATIONS

- **Non-deterministic evaluation.** Three evaluations of one saved checkpoint with the same seed argument gave coverage between 24.6% and 37.8%, and proximity-event counts of 0 to 19; the cause was not isolated: greedy actions remove policy sampling noise, and we suspect simulator or runtime timing as a contributor, but physics scheduling, message ordering and reset state were not separately tested. The three repeats used one checkpoint on the cafe for 150 steps. Single-episode cells (Tables II and III) are descriptive only.
- **Soft safety margin; no formal guarantee.** See Sec. III-C. We did not measure how often or how much the override rules and filter change each controller’s commands, so reported outcomes are those of controller plus safety stack, not of the controller alone.
- **ACR is a discovered-map metric.** Its denominator excludes only cells where a LiDAR return has registered an obstacle, so values are not directly comparable across controllers that discover obstacles at different rates, and 100% means all currently non-obstacle cells were visited, not that all physically navigable area was verified. A ground-truth-denominator ACR is future work.
- **Transfer to other team sizes is untested.** The encoder accepts up to nine neighbors and is order-invariant, but no trained policy was evaluated with a different number of robots. The sweep uses the classical controller.
- **Ablations not run.** We did not ablate the CBF, the consensus merge, the behavior tree or the scan matcher, so we cannot say how much each contributes.
- **Scope.** Simulation only, one robot model, three robots in the learned experiments, one training world, short training (about 3×10^4 steps), and no comparison with other published coverage planners.

VI. CONCLUSION

MARS provides a documented testbed with archived code and per-run results. In the reported long reference runs the classical controller’s discovered-map ACR was high in four of five worlds, coverage generally grew with team size in the exploratory sweep, and in the single-episode matrix the observed MAPPO discovered-map ACR was lower than the classical controller’s in all five worlds. These results are descriptive: evaluation is not deterministic, the learned and classical controllers differ in information and design, the safety margin is soft, and several architectural claims remain untested. We release the code, per-run logs and scripts so that others can check and extend them.

Environment.: ROS 2 Jazzy, Gazebo Harmonic, Ubuntu 24.04.5 LTS, AMD Ryzen 5 7600 (6 cores, 12 threads), about 30 GB RAM, Ray RLlib 2.55.1 (PyTorch, ModelV2 API stack), OSQP, PettingZoo parallel API; Gazebo seed 42.

AI assistance disclosure.: AI tools (Anthropic’s Claude) were used during the project for code and analysis-script assistance and for language editing and structural critique of this manuscript. The author reviewed and verified the results against the archived data and is solely responsible for the methodology, experiments, analysis and conclusions.

Data availability.: Code, per-run results and scripts are in the project repository, archived at <https://doi.org/10.5281/zenodo.22758286> (directory checkpoints/); the repository state described here is Git commit `7c41a00`, and the experiments were run at earlier development commits; GitHub: <https://github.com/harsh-pandhe/MARS>. Raw traces of the two one-off hazard scenarios (Sec. IV) are not archived.

REFERENCES

- [1] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *IEEE International Symposium on Computational Intelligence in Robotics and Automation*, 1997.
- [2] W. Burgard, M. Moors, C. Stachniss, and F. E. Schneider, “Coordinated multi-robot exploration,” *IEEE Transactions on Robotics*, vol. 21, no. 3, pp. 376–386, 2005.
- [3] F. Amigoni, J. Banfi, and N. Basilico, “Multirobot exploration of communication-restricted environments: A survey,” *IEEE Intelligent Systems*, vol. 32, no. 6, pp. 48–57, 2017.
- [4] H.-L. Choi, L. Brunet, and J. P. How, “Consensus-based decentralized auctions for robust task allocation,” *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.
- [5] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 611–24 624.
- [6] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica, “RLlib: Abstractions for distributed reinforcement learning,” in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 3053–3062.
- [7] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, “Deep sets,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [8] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [9] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, “End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks,” in *AAAI Conference on Artificial Intelligence*, 2019.
- [10] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [11] H. Pandhe, “Diagnosing a low-displacement failure mode in MAPPO for multi-robot area coverage,” SSRN preprint, abstract id 7556699; code and data: <https://doi.org/10.5281/zenodo.22758286>, 2026.
- [12] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [13] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
- [14] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. Santos, R. Perez, C. Horsch, C. Dieffendahl, N. L. Williams, Y. Lokesh, and P. Ravi, “PettingZoo: Gym for multi-agent reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.